

Introduction.....	
Architecture.....	
Les composants du système.....	
Schéma simplifié du flux.....	
Installation du système d'exploitation.....	
Outil utilisé : Raspberry Pi Imager.....	
Choix de l'OS : Raspberry Pi OS Lite (64-bit).....	
Configuration avant le flash.....	
Première connexion SSH.....	
Mise à jour du système.....	
Scan réseau.....	
Outils utilisés : arp-scan et nmap.....	
Premier scan du réseau.....	
Note importante : la discrétion des scans.....	
Limites de l'identification.....	
Bot Telegram — Alertes en temps réel.....	
Création du bot.....	
Récupération du Chat ID.....	
Script d'alerte Python.....	
Persistance de la liste des appareils.....	
Mise en service automatique.....	
Suricata — Détection d'intrusion.....	
Qu'est-ce qu'un IDS ?.....	
Installation.....	
Mise à jour des règles.....	
Configuration.....	
Alertes Telegram.....	
Mise en service automatique.....	
Dashboard web — Visualisation et Playbook.....	
Outil utilisé : Flask.....	
Accès au dashboard.....	
Contenu du dashboard.....	
Mise en service automatique.....	
Fail2ban — Protection SSH automatique.....	
Qu'est-ce que Fail2ban ?.....	
Installation.....	
Configuration.....	
Alertes Telegram.....	
Test du système.....	
Vérification du statut.....	
Conclusion.....	
Glossaire.....	

Introduction

Ce projet est né d'une envie simple : comprendre ce qui se passe sur mon réseau Wi-Fi à la maison. Qui s'y connecte ? Y a-t-il des appareils inconnus ? Des tentatives d'intrusion ? Et surtout, comment réagir efficacement si quelque chose de suspect se produit ?

Dans un monde où nos maisons sont de plus en plus connectées, téléphones, ordinateurs, télévisions, consoles de jeux, il devient essentiel de savoir ce qui circule sur notre réseau. La plupart des gens n'ont aucune visibilité là-dessus, et c'est exactement ce problème que ce projet cherche à résoudre.

Pour y répondre, j'ai transformé un Raspberry Pi 5, un mini-ordinateur de la taille d'une carte de crédit, en une véritable sonde de surveillance réseau. Baptisé **Sentinelle** 🦋, ce système tourne 24h/24 en arrière-plan, analyse le trafic du réseau domestique, détecte les comportements suspects et envoie des alertes en temps réel directement sur Telegram.

Ce document retrace chaque étape de la construction de ce système, des choix techniques aux commandes utilisées, avec des explications accessibles même sans connaissances approfondies en informatique.

Matériel utilisé :

- Raspberry Pi 5 (16GB RAM) avec boîtier officiel, alimentation 27W et carte SD
- Atolla USB 3.0 Hub
- Raspberry Pi Camera Module 3 NoIR Wide (prévu pour une future évolution)
- Un réseau Wi-Fi domestique SFR

Architecture

Avant de rentrer dans les détails techniques, voici une vue d'ensemble du système Sentinelle et de comment ses différentes parties s'articulent.

Le Raspberry Pi est le cœur du dispositif. Il est connecté au réseau Wi-Fi domestique et joue le rôle de point d'observation central. Tous les outils installés dessus travaillent ensemble pour surveiller, analyser et alerter.

Les composants du système

arp-scan et nmap sont les outils de découverte réseau. Ils permettent de lister tous les appareils connectés au réseau Wi-Fi en interrogeant les adresses IP et MAC. C'est grâce à eux que Sentinelle sait combien d'appareils sont présents et peut détecter l'arrivée d'un inconnu.

Suricata est le moteur de détection d'intrusion. Il analyse en temps réel le trafic réseau qui passe par le Pi et le compare à une base de plus de 50 000 règles de sécurité connues. Si quelque chose correspond à une menace connue, il génère une alerte.

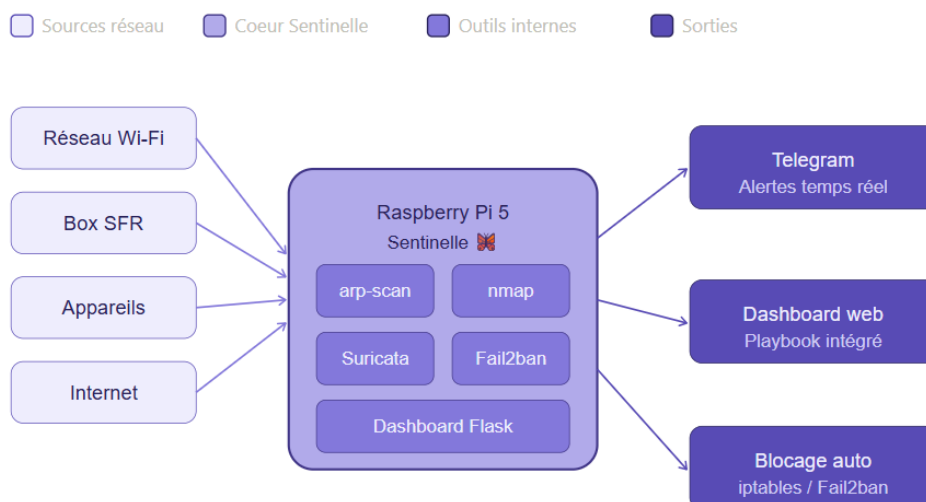
Le bot Telegram est le système de notification. Dès qu'une anomalie est détectée, un message est envoyé automatiquement sur le téléphone via l'application Telegram.

Le Dashboard Flask est l'interface de visualisation. Accessible depuis n'importe quel navigateur sur le réseau local, il affiche les dernières alertes Suricata et le playbook de réponse aux incidents.

Fail2ban est le gardien du SSH. Il surveille les tentatives de connexion ratées et bannit automatiquement les adresses IP suspectes.

Schéma simplifié du flux

Réseau Wi-Fi → Raspberry Pi Sentinelle → Analyse (Suricata + arp-scan) → Alertes (Telegram + Dashboard)



Installation du système d'exploitation

La première étape du projet consiste à installer un système d'exploitation sur la carte microSD du Raspberry Pi. C'est ce système qui va faire tourner tous les outils de surveillance.

Outil utilisé : Raspberry Pi Imager

Raspberry Pi Imager est le logiciel officiel fourni par la fondation Raspberry Pi. Il permet de télécharger et d'écrire automatiquement un système d'exploitation sur une carte microSD depuis un PC Windows, sans manipulation complexe.

Choix de l'OS : Raspberry Pi OS Lite (64-bit)

Parmi les versions disponibles, j'ai choisi la version "Lite" 64-bit pour deux raisons principales. D'abord, elle ne contient pas d'interface graphique, ce qui la rend beaucoup plus légère (environ 550 Mo contre 1,9 Go pour la version complète). Ensuite, toutes les ressources du Pi sont ainsi réservées aux outils de surveillance plutôt qu'à l'affichage d'un bureau inutile. Le Pi sera contrôlé entièrement à distance via SSH.

Configuration avant le flash

Raspberry Pi Imager permet de préconfigurer le système avant même de l'écrire sur la carte, grâce au menu de personnalisation. Voici les paramètres renseignés :

Paramètre	Valeur choisie
Nom d'hôte	sentinelle
Nom d'utilisateur	sentinel
Localisation	Paris / Europe/Paris
Type de clavier	fr
Réseau Wi-Fi	SFR_CF00 (2.4GHz)
SSH	Activé avec mot de passe

Le choix du réseau 2.4GHz plutôt que le 5GHz est délibéré : il offre une meilleure portée à travers les murs, ce qui est préférable pour un appareil fixe qui tourne en permanence.

Première connexion SSH

Une fois la carte flashée et insérée dans le Pi, il suffit de brancher l'alimentation. Après environ 2 minutes de démarrage, le Pi est accessible depuis n'importe quel ordinateur du réseau via la commande suivante :

```
ssh sentinel@sentinelle.local
```

SSH (Secure Shell) est un protocole qui permet de contrôler un ordinateur à distance via une ligne de commande sécurisée, comme si on était physiquement devant lui.

Mise à jour du système

La première commande exécutée après connexion est la mise à jour complète du système :

```
sudo apt update && sudo apt upgrade -y
```

Scan réseau

Une fois le Raspberry Pi opérationnel, la première fonctionnalité mise en place est la découverte des appareils connectés au réseau. L'objectif est simple : savoir à tout moment qui est connecté sur le Wi-Fi de la maison.

Outils utilisés : arp-scan et nmap

Deux outils complémentaires ont été installés pour cette tâche :

```
sudo apt install nmap arp-scan -y
```

arp-scan est un outil qui envoie des requêtes ARP sur le réseau local et liste tous les appareils qui répondent, avec leur adresse IP et leur adresse MAC. L'adresse MAC est un identifiant unique attribué à chaque carte réseau : c'est grâce à elle qu'on peut reconnaître un appareil même si son adresse IP change.

nmap (Network Mapper) est un outil plus complet qui permet non seulement de découvrir des appareils, mais aussi de scanner leurs ports ouverts et d'identifier leur système d'exploitation. C'est l'un des outils les plus utilisés en cybersécurité.

Premier scan du réseau

La commande suivante liste tous les appareils connectés au réseau :

```
sudo arp-scan --localnet --ouifile=/tmp/ieee-oui.txt
```

Le résultat affiche pour chaque appareil son adresse IP, son adresse MAC et le fabricant de la carte réseau (Apple, Samsung, SFR...). Lors du premier scan, 5 appareils ont été détectés sur le réseau domestique.

Note importante : la discrétion des scans

Lors des tests, le logiciel de sécurité ESET installé sur le PC a déclenché une alerte lors d'un scan nmap agressif (`nmap -A`). Cela démontre que certains types de scans sont détectables et considérés comme suspects. Pour la surveillance passive, on privilégie donc des scans moins intrusifs comme `nmap -sn` qui se contente d'envoyer des pings sans scanner les ports.

Limites de l'identification

Les téléphones récents (iPhone iOS 14+, Android 10+) utilisent la randomisation MAC : ils changent volontairement leur adresse réseau pour protéger la vie privée. Il est donc impossible de les identifier de façon fiable via leur adresse MAC uniquement. La box SFR reste la meilleure source pour identifier ces appareils, car elle conserve les noms d'hôtes attribués lors de la connexion.

Bot Telegram — Alertes en temps réel

Pour que Sentinelle soit vraiment utile au quotidien, il faut qu'elle puisse prévenir immédiatement sur le téléphone dès qu'un événement suspect se produit. Le choix s'est porté sur Telegram pour plusieurs raisons pratiques : son API est gratuite, simple à utiliser depuis un script Python, et les messages arrivent instantanément sans risque de finir dans les spams contrairement aux emails.

Création du bot

La création d'un bot Telegram se fait en quelques minutes directement depuis l'application, via un bot officiel appelé @BotFather. Les étapes sont les suivantes : ouvrir une conversation avec @BotFather, taper la commande `/newbot`, choisir un nom, puis récupérer le token d'accès généré automatiquement.

Le bot créé pour ce projet s'appelle **SentinelleLibellule_Bot** 🦋

Récupération du Chat ID

Pour que le bot sache à qui envoyer les messages, il faut récupérer son Chat ID, c'est-à-dire l'identifiant unique de la conversation. Après avoir envoyé un premier message au bot depuis Telegram, la commande suivante permet de le récupérer :

```
curl "https://api.telegram.org/botTOKEN/getUpdates"
```

La réponse JSON contient le champ "id" correspondant au Chat ID.

Script d'alerte Python

Le script principal de surveillance tourne en permanence sur le Pi. Il scanne le réseau toutes les 30 minutes et envoie une alerte Telegram dès qu'un nouvel appareil inconnu apparaît :

```
import requests
import subprocess
import time
import json
import os

TOKEN = "token_du_bot"
CHAT_ID = "identifiant_conversation"

def envoyer_alerte(message):
    url = f"https://api.telegram.org/bot{TOKEN}/sendMessage"
    requests.post(url, json={"chat_id": CHAT_ID, "text": message})

def scanner_reseau():
    result = subprocess.run(
        ["sudo", "arp-scan", "--localnet", "--ouifile=/tmp/ieee-oui.txt"],
```



```

        capture_output=True, text=True
    )
    appareils = set()
    for ligne in result.stdout.split("\n"):
        if "192.168." in ligne:
            parts = ligne.split()
            if len(parts) >= 3:
                appareils.add((parts[0], parts[1], parts[2]))
            elif len(parts) == 2:
                appareils.add((parts[0], parts[1], "Inconnu"))
    return appareils

while True:
    appareils = scanner_reseau()
    for appareil in appareils:
        if appareil not in appareils_connus:
            appareils_connus.add(appareil)
            envoyer_alerte(f"⚠️ Nouvel appareil détecté !\nIP: {appareil[0]}\nMAC: {appareil[1]}\nFabricant: {appareil[2]}")
    time.sleep(1800)

```

Persistence de la liste des appareils

Pour éviter de recevoir une alerte à chaque redémarrage du Pi pour des appareils déjà connus, la liste est sauvegardée dans un fichier JSON sur la carte SD. Ainsi, même après un reboot, Sentinelle se souvient des appareils déjà vus.

Mise en service automatique

Le script est configuré comme un service systemd, ce qui signifie qu'il démarre automatiquement au démarrage du Pi, même sans intervention humaine :

```

sudo systemctl enable sentinelle
sudo systemctl start sentinelle

```

Suricata — Détection d'intrusion

Le script de surveillance arp-scan détecte les nouveaux appareils, mais il ne voit pas ce qui se passe dans le trafic réseau lui-même. Pour aller plus loin, on installe **Suricata**, un IDS (Intrusion Detection System), c'est-à-dire un système de détection d'intrusion.

Qu'est-ce qu'un IDS ?

Un IDS analyse en temps réel les paquets réseau qui transitent par l'interface réseau du Pi et les compare à une base de règles de sécurité connues. Si un paquet correspond à une signature d'attaque connue, une alerte est générée. C'est l'équivalent d'un antivirus, mais pour le trafic réseau.

Installation

```
sudo apt install suricata -y
```

Mise à jour des règles

Suricata fonctionne avec des règles de détection maintenues par la communauté. La commande suivante télécharge automatiquement les règles Emerging Threats Open, une base reconnue dans le monde de la cybersécurité :

```
sudo suricata-update
```

Au total, **50 497 règles** ont été chargées. Ces règles couvrent des milliers de menaces connues : scans de ports, tentatives d'exploitation, malwares, trafic suspect, et bien d'autres.

Configuration

Suricata est configuré pour surveiller l'interface Wi-Fi `wlan0` du Pi. Sa configuration principale se trouve dans le fichier `/etc/suricata/suricata.yaml`.

Une règle personnalisée a été ajoutée pour les tests dans le fichier

```
/var/lib/suricata/rules/test.rules:  
alert icmp any any -> any any (msg:"Test ping detecte"; sid:9000001;  
rev:1;)
```

Cette règle déclenche une alerte à chaque fois qu'un ping est détecté sur le réseau — utile pour vérifier que Suricata fonctionne correctement.

Alertes Telegram

Un second script Python surveille en permanence le fichier de logs de Suricata (/var/log/suricata/eve.json) et envoie une alerte Telegram dès qu'une nouvelle alerte est détectée :

```
if event.get("event_type") == "alert" and "Telegram" not in
event.get("alert", {}).get("signature", ""):
    envoyer_alerte(f"🚨 Alerte Suricata !\nIP source:
{event.get('src_ip')}\nIP dest: {event.get('dest_ip')}\nSignature:
{event['alert'].get('signature')}\nSévérité:
{event['alert'].get('severity')}")
```

Le filtre "Telegram" not in signature est important : sans lui, Suricata génère des alertes en boucle sur ses propres communications avec l'API Telegram, ce qui noie les vraies alertes.

Mise en service automatique

Comme pour le script de surveillance réseau, Suricata et son script d'alerte sont tous les deux configurés comme services systemd :

```
sudo systemctl enable suricata
sudo systemctl enable suricata-alerte
```

Dashboard web — Visualisation et Playbook

Recevoir des alertes sur Telegram est utile, mais il manquait un endroit centralisé pour visualiser l'historique des alertes et savoir quoi faire en cas d'incident. C'est le rôle du dashboard.

Outil utilisé : Flask

Flask est un framework Python léger qui permet de créer des applications web facilement. Il a été choisi car il est simple à mettre en place, ne nécessite pas de serveur web complexe, et s'intègre naturellement avec les scripts Python déjà en place.

```
sudo apt install python3-flask -y
```

Accès au dashboard

Le dashboard est accessible depuis n'importe quel navigateur connecté au réseau Wi-Fi domestique, à l'adresse suivante :

```
http://192.168.1.26:5000
```

Aucune installation n'est nécessaire côté utilisateur — il suffit d'ouvrir un navigateur.

Contenu du dashboard

Le dashboard se compose de deux parties principales.

La première partie affiche les **20 dernières alertes Suricata** en temps réel, avec pour chaque alerte l'adresse IP source, l'adresse IP de destination, le nom de la signature déclenchée et l'horodatage. La page se rafraîchit automatiquement toutes les 30 secondes.

La seconde partie est le **Playbook de réponse aux incidents**. C'est une liste d'actions concrètes à mener pour chaque type de menace détectée. L'idée est d'avoir directement sous la main les étapes à suivre sans avoir à chercher quoi faire dans l'urgence.

Menace détectée	Actions à mener
Nouvel appareil inconnu	Identifier le MAC, croiser avec la liste blanche, isoler si suspect, changer le mot de passe Wi-Fi
Scan de ports	Identifier l'IP source, déterminer si interne ou externe, bloquer via iptables, logger l'incident
Tentative de brute force	Identifier le service ciblé, laisser Fail2ban agir, alerter, documenter
Trafic anormal	Capturer avec tcpdump, analyser le protocole, identifier la source, décider du blocage

Mise en service automatique

Le dashboard est lui aussi configuré comme un service systemd pour tourner en permanence :

```
sudo systemctl enable dashboard
sudo systemctl start dashboard
```

Fail2ban — Protection SSH automatique

Sentinelle étant accessible à distance via SSH, il est important de la protéger contre les tentatives de connexion non autorisées. Un attaquant pourrait en effet tenter de deviner le mot de passe en essayant des milliers de combinaisons automatiquement, c'est ce qu'on appelle une attaque par brute force. Fail2ban est l'outil qui permet de contrer ce type d'attaque.

Qu'est-ce que Fail2ban ?

Fail2ban surveille en permanence les fichiers de logs du système et détecte les tentatives de connexion échouées répétées. Dès qu'une adresse IP dépasse un seuil défini, elle est automatiquement bannie via une règle iptables, c'est-à-dire que le Pi refuse tout simplement de lui répondre pendant une durée déterminée.

Installation

```
sudo apt install fail2ban -y
```

Configuration

La configuration se fait dans le fichier `/etc/fail2ban/jail.local` :

```
[DEFAULT]
bantime = 1h
findtime = 10m
maxretry = 3

[sshd]
enabled = true
port = ssh
logpath = /var/log/auth.log
maxretry = 3
```

Ces paramètres signifient concrètement : si une adresse IP échoue 3 fois à se connecter en SSH en moins de 10 minutes, elle est bannie pendant 1 heure.

Alertes Telegram

Comme pour les autres composants, Fail2ban a été connecté au bot Telegram. Une notification est envoyée automatiquement à chaque bannissement et débannissement d'une adresse IP. La configuration se trouve dans le fichier `/etc/fail2ban/action.d/telegram.conf` :

```
[Definition]
```

```
actionban = curl -s "https://api.telegram.org/botTOKEN/sendMessage?chat_id=CHAT_ID&text=🚫 IP <ip> bannie ! Jail: <name>"
```

```
actionunban = curl -s "https://api.telegram.org/botTOKEN/sendMessage?chat_id=CHAT_ID&text=✅ IP <ip> débannie. Jail: <name>"
```

Test du système

Pour vérifier le bon fonctionnement, un test a été réalisé en entrant volontairement un mauvais mot de passe SSH trois fois de suite. Le résultat a été immédiat : l'adresse IP du PC a été bannie et une notification Telegram a été reçue avec le message "🚫 IP bannie par Fail2ban !".

Pour débannir manuellement une adresse IP depuis une session SSH déjà ouverte :

```
sudo fail2ban-client set sshd unbanip ADRESSE_IP
```

Vérification du statut

À tout moment, il est possible de consulter l'état de Fail2ban et la liste des IPs bannies :

```
sudo fail2ban-client status sshd
```

Conclusion

Ce projet a permis de construire de A à Z une sonde de surveillance réseau fonctionnelle, baptisée **Sentinelle** 🐛, à partir d'un simple Raspberry Pi 5 et d'outils open source.

Ce que Sentinelle est capable de faire

A l'issue de ce projet, le système surveille le réseau domestique en permanence et est capable de détecter l'arrivée d'un nouvel appareil inconnu sur le Wi-Fi, d'analyser le trafic réseau à la recherche de comportements suspects grâce à plus de 50 000 règles de détection, de bloquer automatiquement les tentatives de connexion SSH répétées, d'envoyer des alertes en temps réel sur Telegram pour chacun de ces événements, et de centraliser toutes ces informations dans un dashboard web accessible depuis n'importe quel navigateur du réseau.

Ce que ce projet m'a appris

Au delà des aspects techniques, ce projet a été une introduction concrète à plusieurs concepts fondamentaux de la cybersécurité. La mise en place de Suricata a permis de comprendre le fonctionnement d'un IDS et l'importance des règles de détection. La configuration de Fail2ban a illustré la notion de défense en profondeur. Et la création du playbook a mis en lumière l'importance d'avoir des procédures claires et documentées avant qu'un incident ne se produise, plutôt que d'improviser dans l'urgence.

La mésaventure avec le fichier de configuration de Suricata, qui a nécessité une réinstallation complète suite à une corruption du fichier YAML, a également été une leçon précieuse : toujours sauvegarder les fichiers de configuration avant de les modifier.

Axes d'amélioration

Ce projet est une base solide qui peut évoluer dans plusieurs directions. Connecter le Pi en câble Ethernet directement sur la box SFR permettrait à Suricata de surveiller l'intégralité du trafic réseau et non uniquement celui du Pi lui-même. Intégrer la caméra NoIR Wide permettrait d'ajouter une dimension de surveillance physique avec détection de mouvement. Mettre en place un système de logs centralisés avec des graphiques de tendance enrichirait le dashboard. Enfin, automatiser les réponses aux incidents, comme le blocage automatique d'une IP suspecte dès la première alerte Suricata, renforcerait encore le niveau de protection.

Mot de fin

Sentinelle est un projet qui prouve qu'il est possible de mettre en place des outils de sécurité sérieux à la maison, avec peu de matériel et sans budget conséquent. C'est aussi une excellente façon d'apprendre la cybersécurité de façon pratique, en se confrontant aux vraies problématiques du terrain plutôt qu'en restant dans la théorie.

Glossaire

ARP (Address Resolution Protocol) : protocole réseau qui permet de faire le lien entre une adresse IP et une adresse MAC sur un réseau local.

Brute force : technique d'attaque qui consiste à essayer automatiquement un très grand nombre de combinaisons de mots de passe jusqu'à trouver le bon.

CHAT ID : identifiant unique d'une conversation Telegram, nécessaire pour qu'un bot sache à qui envoyer ses messages.

IDS (Intrusion Detection System) : système de détection d'intrusion qui analyse le trafic réseau et génère des alertes en cas de comportement suspect.

IP (Internet Protocol) : adresse numérique unique attribuée à chaque appareil connecté à un réseau, permettant de l'identifier et de lui envoyer des données.

iptables : outil Linux qui permet de définir des règles de filtrage du trafic réseau, notamment pour bloquer des adresses IP.

JSON (JavaScript Object Notation) : format de fichier texte léger utilisé pour stocker et échanger des données de façon structurée et lisible.

MAC (Media Access Control) : adresse physique unique gravée dans la carte réseau de chaque appareil, utilisée pour l'identifier sur un réseau local.

OS (Operating System) : système d'exploitation, c'est-à-dire le logiciel de base qui fait fonctionner un ordinateur (ici Raspberry Pi OS).

Playbook : document listant les actions précises à mener en réponse à un type d'incident de sécurité donné.

Port : point d'entrée numérique sur un appareil réseau. Chaque service utilise un port spécifique (le port 22 pour SSH, le port 80 pour HTTP, etc.).

Raspberry Pi : mini-ordinateur monocarte de la taille d'une carte de crédit, conçu pour des projets électroniques et informatiques.

Règle Suricata : instruction qui décrit une signature de menace connue. Si le trafic réseau correspond à cette signature, une alerte est déclenchée.

Script Python : fichier contenant une série d'instructions écrites en langage Python, exécutées automatiquement par le Pi.

Service systemd : programme configuré pour démarrer automatiquement au lancement du système d'exploitation, sans intervention humaine.

SSH (Secure Shell) : protocole permettant de se connecter à distance à un ordinateur via une ligne de commande sécurisée et chiffrée.

Token : clé secrète unique générée par Telegram pour authentifier un bot et lui permettre d'envoyer des messages via l'API.

YAML : format de fichier texte utilisé pour les fichiers de configuration, conçu pour être lisible par les humains.